

PhishLens

A phishing-detection system that runs entirely on the machine it protects.
How it works, what it is built from, and what it does not do.

Version	1.7.10 (desktop application)
Document date	22 September 2026
Repository	github.com/mdyounus-git/PhishLens
Composition	54 backend modules, 14 adapters, 12 detection rules, 23 console panels
Test suite	323 backend, 33 desktop, 4 console checks - all passing
Runtime	Node.js 24.11.1, Electron 44.4.3, Chrome extension (Manifest V3)
Cost to run	Nothing. No paid service, no subscription, no account with us.

The one sentence version

PhishLens reads mail before you open it, gathers facts about it from several independent angles - who really sent it, what the language is doing, what is inside the attachments, where the infrastructure sits, whether anything like it has been seen here before - then combines those facts into a score it can fully explain, and acts on the result. Every part of that happens on your own computer.

This document is written to be read start to finish by somebody who has not seen the code. Sections 1 to 4 explain what the system is and how a message travels through it. Sections 5 to 9 go into what it actually looks for. Sections 10 to 14 cover the technology, the data, and how to run it. Section 15 states honestly what is finished and what is not.

Contents

1	The problem, and the shape of the answer
2	The four pieces, and how they fit together
3	Seven ways mail gets in
4	The pipeline: what happens to one message
5	Evidence fusion and the confidence score
6	What it actually looks for
7	Inside the attachments
8	The detection rules
9	Optional open-source tools, and the cost of not having them
10	After the verdict: policy, remediation, audit
11	The console
12	The browser extension
13	The technology stack, in full
14	Your data: where it lives and what leaves the machine
15	Running it, testing it, and what is still open
16	Glossary

1. The problem, and the shape of the answer

Phishing filters were built in an era when a phishing email announced itself. The spelling was wrong, the grammar was strange, the greeting was generic. Keyword lists worked because attackers wrote badly.

That era is over. An attacker with a language model writes a message in flawless English, in your company's house style, referring to a real project, signed off by a real colleague. There is no misspelling to catch. A detector that scores messages on how they read is now scoring the one thing the attacker has completely solved.

PhishLens is built on the opposite assumption: assume the prose is perfect, and look at everything else. The things an attacker cannot fake as easily are structural and factual, and they do not improve when the writing does:

- **Who really sent it.** SPF, DKIM and DMARC are cryptographic and administrative facts. A convincing sentence does not make a DKIM signature verify.
- **Where it physically came from.** The relay chain, reconstructed with a view of which hops can be trusted, points at an origin address. That address sits in a netblock with a reputation, a hosting profile, and possibly a VPN or Tor exit record.
- **What the thread claims to be.** In a hijacked conversation the prose is genuinely perfect - it is a real conversation. What is wrong is structural: the reply arrives from somewhere the thread never involved, or continues a thread this installation has never seen.

- **What is inside the attachments.** A macro that runs on open, an executable renamed to look like a document, a filename reversed with a direction-override character. These are byte-level facts.
- **Whether the message is hiding from being read.** Zero-width characters between letters, lookalike Cyrillic glyphs, a body that is one obfuscated script. Ordinary mail does not hide characters inside words.
- **Whether anything like it has been here before.** Shared infrastructure, repeated senders, a campaign forming across cases that individually looked unremarkable.

Language analysis is still present, and it is still useful - but it is one family of evidence among eleven, and it is capped so that it can never decide a case on its own. That cap is the whole design philosophy in one number.

The commitments this was built under

Commitment	What it means in practice
Free, permanently	No paid API, no commercial threat feed, no subscription tier. Everything used is open source or a free public service. There is no plan under which this becomes paid.
Nothing depends on our servers	Install it and disconnect the developer entirely - it keeps working. Your mail, your cases, your keys and your history are files on your disk. We operate no service that your copy talks to.
Never report unchecked as clean	If a check could not run - a lookup timed out, a tool is missing, a file could not be read - the result says so. It does not fall back to "looks fine". A silent failure that reports SAFE is worse than no product.
Every verdict is explainable	The score is not a model output. It is a sum of named findings with stated weights, and the breakdown is in the case file. You can always answer "why did it say that".
Two themes	Light and dark, switchable in the console, remembered between sessions.

2. The four pieces, and how they fit together

PhishLens is one system in four parts. Only the first is essential; the others are ways of getting mail into it and seeing what it found.

Part	What it is	What it does
Backend	A Node.js service (Express 5) listening on localhost.	Everything that matters. Parsing, all detection, scoring, storage, remediation, the REST API. 54 modules and 14 adapters.
Console	A React 19 single-page application built with Vite.	23 panels: the threat feed, case investigation, the evidence graph, the global map, remediation posture, audit timeline, detection configuration, reports.
Desktop app	Electron 44, packaged as a Windows installer.	Wraps the two above so there is nothing to configure. Starts the backend as a supervised child process, serves the console, manages the data directory and the API key.
Browser extension	Manifest V3, for Chrome and Edge.	Watches Gmail and Outlook Web. Reads messages from the list using the session already signed in, so no password or token is ever stored.

How they connect

Sources	Transport	Note
SMTP relay	port 2525	Inline gateway
Chrome / Edge	extension	Reads webmail in place
IMAP mailbox	poller	For a mailbox nothing else reaches
Gmail API	OAuth + Pub/Sub	Push notification, then history pull
REST / webhook / file	HTTP	Integrations and analyst submissions

All of them converge on the same place. There is one analysis pipeline, not one per channel - a message that arrives by SMTP is judged by exactly the same code, in the same order, as a message the extension read out of Gmail. The only thing the channel changes is how much of the original message is available: the extension sometimes has only what is visible in the list, and when that happens the case is explicitly marked `BODY_ONLY` so nothing downstream treats a partial read as a full header analysis.

Why the desktop app exists

The backend and console can be run by hand with two terminal windows. The Electron app exists so that they do not have to be. It generates the API key on first run, points the console at the right port, keeps the backend alive if it crashes, and puts the data somewhere Windows will not clear. Nothing in the desktop layer performs detection - remove it and the system is unchanged.

3. Seven ways mail gets in

The system keeps a registry of every channel it knows about, not only the ones switched on - so the console can show you what is *not* being watched. Three are always available to any authenticated caller; four need setting up.

Channel	Transport	What it does	State
Inline SMTP gateway	SMTP	Mail delivered straight to the PhishLens listener, typically from an MTA relay or milter at the network boundary. This is the earliest possible interception point - the message is judged before it is delivered.	Active verified end to end
Browser (webmail)	BROWSER	The extension examines messages in Gmail and Outlook Web as they appear in the list, before they are opened. It fetches the original message using the browser session that is already signed in.	Installed not yet reporting
IMAP mailbox poller	IMAP	Polls a mailbox for newly delivered messages. Kept for a mailbox that nothing else can reach; the extension and the SMTP gateway both see mail earlier.	Needs an app password
Gmail API	OAuth	Receives Gmail push notifications over Pub/Sub and pulls the new messages through the history API.	Needs a Google Cloud OAuth client
REST analysis API	HTTP	Authenticated POST /api/analyze - used by integrations and for samples an analyst submits directly.	Always on
Inbound webhook	HTTP	Authenticated POST /api/ingest/email, for a SIEM, SOAR, ticketing system or mail relay pushing messages in.	Always on
Message file upload	HTTP	Authenticated POST /api/ingest/file, taking .eml and .msg files - for user-reported phishing and analyst samples.	Always on

Heartbeats, and why they matter

Two of these channels are expected to check in: the browser watcher every 15 minutes, the IMAP poller every minute. If a heartbeat stops arriving, the registry marks the channel degraded rather than leaving it showing green. This is the "never report unchecked as clean" commitment applied to the plumbing - a monitoring system that silently stops monitoring is the most dangerous failure it can have, because it looks identical to an inbox with no phishing in it.

Deduplication

The same message can legitimately arrive through more than one channel - the SMTP gateway sees it in transit and the extension sees it in the inbox. Before any analysis runs, a message key is reserved from a persistent deduplication store, keyed on the RFC message-id and a SHA-256 of the raw bytes. A second arrival returns the existing case rather than creating a duplicate, and a message still being processed is rejected rather than analysed twice in parallel.

4. The pipeline: what happens to one message

This is the actual execution order in the backend, not a simplification. The order is dictated by real data dependencies - each stage needs something an earlier one produced.

Preprocessing: establishing the facts

#	Stage	What it establishes
1	MIME parsing	Headers, body parts, attachments - parsed by PhishLens itself, with no external service involved.
2	Optional external provider	A hook for a supplementary detection service. Defaults to skipped, and is never a hard dependency.
3	ThreatObject created	One canonical object that every later stage adds to. Provenance is recorded first, so remediation later resolves a real target instead of a default - a message with no provenance is reported as "not actionable" rather than "action failed".
4	SPF / DKIM / DMARC	Authentication analysed independently rather than trusting the receiving server's own header.
4b	ARC chain (RFC 8617)	Recorded as context, never as reassurance. A valid chain says who handled the message, not whether they can be trusted. Its one use is distinguishing an authentication failure caused by ordinary forwarding from one caused by spoofing.
5	Attachment metadata	Names, declared types, sizes, hashes. Nothing is executed at any point.
5b	QR codes decoded	Deliberately before indicator extraction. A link inside a QR image is invisible to everything that reads text; recovering it here means the feeds, domain-age and lookalike checks all apply to it exactly as they would to a typed link.
6	Indicator extraction	IPs, domains, URLs, file hashes. The QR links join this set rather than sitting in a field nothing reads.

Detection: reading the message itself

#	Stage	What it establishes
6b	Text deception	Runs before any language analysis, because the language analysis reads its output: a zero-width space between every character defeats every substring match, and folding the text back is what makes the patterns applicable at all. The obfuscation itself is the stronger signal - ordinary mail does not hide characters inside words.
6c	Thread integrity	The one check AI fluency cannot help an attacker with. Who the reply actually came from, and whether the thread it claims to continue ever happened in this installation.
7	Explainable NLP	Social-engineering structure - urgency, authority, secrecy, payment redirection. Every signal is named and quotable, never an opaque score.
8	Rule engine (first pass)	The native MQL-style rules that reason over the message alone. Each rule cites its own source.

Four analysis branches

Two of these are genuinely independent of one another and run concurrently. The other two cannot: the forensic branch selects the origin IP that the behavioural and infrastructure branches both reason about, and campaign correlation needs the enriched indicators the infrastructure branch produces.

Branch	What it does
1 - Forensic	Reconstructs the SMTP relay chain with an explicit model of which hops can be trusted, and selects the true origin address. Runs first because the next two depend on its answer.
2 - Behavioural	Compares this message against the baseline this installation has built for that sender: the addresses, relays, times and phrasing normally seen from them. Runs concurrently with branch 3.
3 - Infrastructure and intelligence	Enriches the indicators: domain registration age, hosting and netblock profile, anonymisation (VPN, proxy, Tor, hosting range), reputation, and matches against locally held indicator feeds. Runs concurrently with branch 2.
4 - Campaign correlation	A persistent SQLite graph across every case ever seen here. Shared infrastructure, repeated senders, campaigns forming out of messages that were individually unremarkable.

Convergence: deciding and acting

#	Stage	What it does
8b	Rule engine (second pass)	The rules that reason over what the branches produced - an indicator-feed match, a domain registered four days ago - rather than over the message alone. Detection logic stays in one auditable, cited place instead of being scattered.
9	Adaptive scoring	Characteristics this installation has learned from mail confirmed malicious here. Capped low on purpose - see section 5.
10	Executive protection	Impersonation of, or sustained targeting of, named protected people. Runs before fusion so its findings become evidence through the same path as everything else.
11	Evidence fusion	Eleven sources of findings normalised into one list of evidence objects with a common shape.
12	Confidence engine	The score, and the verdict. Section 5 covers this in full.
13	Policy engine	Decides what should happen, given the verdict and the context.
14	Remediation	Executes that decision - quarantine, alert, or nothing - through the real mailbox connection where one exists.
15	Case stored	Written to disk and bound to the deduplication key, so the same message arriving again returns this case.
16	Post-verdict learning	The sender baseline records what this message looked like, and a corroborated high-risk verdict teaches the adaptive model the characteristics that made it malicious, so later messages sharing them are recognised.

5. Evidence fusion and the confidence score

This is the part worth understanding properly, because it is where PhishLens differs most from a conventional filter, and it is what makes every verdict answerable.

Step one: everything becomes evidence

Eleven different analyses produce findings in eleven different shapes. Fusion normalises all of them into one list of evidence objects, each with a finding, a severity, a confidence, and a type. The eleven sources are: authentication anomalies; return-path mismatch; explainable NLP and social engineering; anonymisation (VPN, proxy, Tor, hosting); reputation; campaign and cross-case correlation; the native rule engine; behavioural comparison against the sender baseline; adaptive learned characteristics; executive protection; and what was found inside the attachments.

Step two: evidence is grouped into families

Findings are sorted into families of related evidence. This matters because SPF failing and DMARC failing are not two independent proofs - DMARC failing is largely a *consequence* of SPF failing. Counting them separately would inflate the score with one fact wearing two hats.

Family	Cap	Why that number
BEC_COMPOSITE	0.35	A composite business-email-compromise pattern is several facts already agreeing.
THREAT_INTEL	0.35	Among the strongest single facts available: somebody has already observed this exact URL, host or netblock being used maliciously.

Family	Cap	Why that number
EXECUTIVE	0.35	Impersonating a named executive is among the most consequential things a message can do.
AUTHENTICATION	0.30	Strong, factual, and occasionally broken by legitimate forwarding.
CUSTOM	0.30	Operator-written rules describe threats this deployment is actually seeing, so they weigh like a built-in family rather than a weak afterthought.
LANGUAGE	0.25	The philosophical centre of the design. Language can raise a case and corroborate one. It can never decide one, because that is exactly the signal an attacker with a language model has already defeated.
CAMPAIGN / IDENTITY / INFRASTRUCTURE / BEHAVIOURAL	0.25	Meaningful corroboration; each needs company to reach a verdict.
URL_RISK / ATTACHMENT_RISK	0.20	Bounded here because the decisive floor below is the proper route for an attachment that is unambiguously malicious.
LEARNED_PATTERN	0.15	Capped lowest, deliberately. What the system taught itself may tip a borderline case, but must never drive a high-risk verdict alone. The deterministic, source-cited rules stay the primary authority.

Step three: within a family, weight halves

Inside one family the strongest finding contributes in full. The next contributes at half that weight, the next at a quarter, and so on. Then the family total is capped.

$$\text{contribution} = \min(v_1 + v_2/2 + v_3/4 + \dots , \text{family cap})$$

Severity sets the base weight before that halving: CRITICAL 0.35, HIGH 0.25, MEDIUM 0.15, LOW 0.05, each multiplied by the finding's own confidence. The effect is that a message with five different *kinds* of problem scores far higher than a message with five symptoms of one problem - which is the correct behaviour, and is not what a flat sum does.

Step four: the decisive floor

Family caps exist to stop correlated facts compounding. They must not also stop a single confirmatory fact from being decisive. A URL currently listed on a malicious-URL feed should not be capped down to "suspicious" merely because nothing else about the message looked unusual.

So certain findings are marked **decisive**: those with no benign interpretation. If any decisive finding is present and the weighted score came out below 0.70, the score is raised to exactly 0.70 - and the raise appears in the breakdown as its own line, naming the finding that caused it. It is a visible floor, never a silent override. You can always see that it happened and why.

Step five: the verdict

Score	Verdict	What it means
≥ 0.70	HIGH_RISK	Acted on. The policy engine decides quarantine or alert.
$0.35 - 0.69$	SUSPICIOUS	Surfaced for a human. Not blocked on its own evidence.
below 0.35	SAFE	Recorded, and used to build the sender baseline. Note that SAFE means "checked and nothing found", never "could not check".

What you get to see

Every case carries the full contribution breakdown: each family, what it contributed, the strongest finding in it, the supporting findings, and a sentence explaining the arithmetic. The scoring version is stamped on the case, so a verdict from an older version is identifiable as such. There is no step in this that you cannot inspect.

6. What it actually looks for

Area	What is examined
Authentication	SPF, DKIM and DMARC evaluated independently, plus the ARC chain read as context. Return-path mismatch against the visible sender.
Relay forensics	The Received chain rebuilt with a trust model, so the origin is the first untrusted hop rather than whatever the last header claims. Relay trust is scored, not assumed.
Identity and lookalikes	Display-name spoofing, lookalike domains against the organisation's own domains, homoglyph substitution, and public-suffix-aware domain comparison (so co.uk is handled correctly).
Text deception	Zero-width and invisible characters inside words, mixed-script text, homoglyphs, direction-override characters. Both flagged in itself and folded back so the language analysis works on the real text.
Language and social engineering	Urgency, authority pressure, secrecy, payment redirection, credential solicitation - each named and quotable. Capped at 0.25, for the reasons in section 5.
Thread integrity	Whether a reply genuinely continues a thread seen here, and whether it came from a party that thread ever involved. The check designed for the hijacked-conversation attack.
Links	Extracted from text, HTML and decoded QR codes alike. Registration age, hosting, redirect structure, mismatch between displayed and real target, and matches against local indicator feeds.
QR codes	Decoded locally from attached and embedded images, and the recovered links treated exactly like typed ones. This closes the gap where a link is invisible to any text-based check.
Attachments	Metadata, hashes and then actual content inspection. Section 7 covers this fully.
Infrastructure	Netblock classification, hosting versus residential, anonymisation (VPN, proxy, Tor, hosting range), reputation, DNS blocklist presence, RDAP registration data.
Campaign graph	A persistent SQLite graph over every case, finding shared infrastructure and repeated actors across messages that individually looked unremarkable.
Behavioural baseline	Per-sender history of addresses, relays, timing and phrasing. A first message from a sender is treated differently from the two-hundredth.
Executive protection	Named protected people and the organisation's domains. Impersonation of them, and sustained targeting of them, both weigh at the top cap.
Adaptive learning	Characteristics learned from mail confirmed malicious in this installation. Capped at 0.15 so it can never outvote the deterministic rules.

7. Inside the attachments

Attachment handling happens in two layers. The first reads metadata - name, declared type, size, hash. The second reads the actual bytes.

Nothing is ever executed

Not the macros, not the scripts, not the documents. The inspector walks file structures and reads bytes. A ZIP central directory is parsed to find vbaProject.bin; a PDF is scanned for active-content markers; an Office file is examined for what would run it automatically. At no point does any attachment code run on your machine, which is the whole point of inspecting it here rather than opening it.

Findings marked decisive

These four have no benign interpretation, so any one of them alone raises the case to HIGH_RISK through the floor described in section 5.

Finding	What it is, and why there is no innocent version
BIDI_FILENAME	A Unicode direction-override character in the filename, used to display "invoice_txt.exe" as "invoice_exe.txt". There is no legitimate reason for a mail attachment to reverse its own name.
DOUBLE_EXTENSION	A name like report.pdf.scr, where the real extension is executable. The first extension exists solely to be the one you read.
EXECUTABLE_DECLARED_AS_DOCUMENT	The declared type says document; the file header says executable. The file is actively lying about what it is.
MACRO_RUNS_AUTOMATICALLY	The macro is bound to an automatic trigger such as AutoOpen or Document_Open. Merely opening the file runs code, with no click and no prompt beyond the macro warning.

Findings that are evidence, not proof

These are deliberately **not** decisive. Each has a real legitimate use, and marking them decisive would manufacture false alarms - which erodes trust in the tool faster than a miss does.

- EXECUTABLE_ATTACHMENT - an executable arrived. Rare in ordinary mail, but developers do send binaries.
- OFFICE_MACROS_PRESENT - the document contains macros. Plenty of genuine business documents do; the decisive version is the one that runs them automatically.
- EXECUTABLE_INSIDE_ARCHIVE - an executable nested inside a ZIP or similar archive, where the container keeps it off the visible attachment name.
- REMOTE_TEMPLATE_REFERENCE - the document pulls a template from a remote address when opened, which is how a clean-looking file fetches its payload after delivery.

PDF active content

PDFs are scanned for seven specific markers rather than being called "suspicious" as a whole, so the case names exactly what was found:

Marker	Meaning	Weight
/OpenAction	Something is set to run as soon as the document opens.	HIGH
/Launch	The document asks to start an external program.	HIGH

Marker	Meaning	Weight
/AA	An action is attached to a page or field event.	MEDIUM
/JavaScript, /JS	The document carries JavaScript.	MEDIUM
/EmbeddedFile	Another file is carried inside this one.	MEDIUM
/SubmitForm	The document can send data to a remote address.	MEDIUM
/RichMedia	The document embeds media that runs a player.	MEDIUM

8. The detection rules

PhishLens carries twelve content rules written in YARA syntax, kept as plain text files you can read and edit. They are evaluated by an engine written for this project, so **the rules work whether or not YARA is installed**. If the real YARA binary is present it is used instead and the case records which engine ran.

The engine implements the subset the rules need: text, hexadecimal and regular-expression strings; the nocase, wide and fullword modifiers; hex wildcards and jumps; and conditions covering "N of them", match counts, offsets, file size and integer reads at an offset.

The rules describe structure, never persuasion

Not one of them looks for a word or a phrase. "Urgent", "verify your account", "your account will be suspended" - none of that appears in any rule. They describe what a file or page is structurally doing: assembling a download in memory, posting credentials offsite, running an encoded command. That is the difference between detection that survives an attacker with a language model and detection that does not.

Rule	What it describes	Decisive
HTML_Smuggling_Blob_Download	The page builds a file in memory and hands it to the browser as a download, so nothing malicious ever crosses the network as a file to be scanned.	Yes
HTML_Credential_Form_Posting_Offsite	A password field in a form that posts to a different origin than the page claims to be.	Yes
HTML_Meta_Refresh_To_Encoded_Page	An automatic redirect to a base64-encoded data URL - a redirect specifically shaped to be unreadable.	Yes
HTML_Clipboard_Paste_To_Run	The "ClickFix" pattern: the page copies a command to your clipboard and instructs you to paste it into a terminal, making you the delivery mechanism.	Yes
LNK_Shortcut_Running_A_Shell	A Windows shortcut file whose target is a command shell.	Yes
Encoded_PowerShell_Command	PowerShell invoked with an encoded command and execution policy bypassed.	Yes
RTF>Loading_Remote_Object	An RTF document that loads an object from a remote address when opened.	Yes
Office_DDE_Auto_Execution	A DDE field set to execute on open - code execution without a macro at all.	Yes
SVG_With_Embedded_Script	An SVG carrying script. Images are not expected to be executable.	No
HTML_Obfuscated_Body_Only	A page whose entire body is obfuscated script and nothing else.	No
Disk_Image_Attachment	An ISO, IMG or VHD attachment - a container historically used to carry executables past mark-of-the-web.	No
Archive_Password_Hint_In_Message	An encrypted archive whose password is written in the message body, which exists to defeat scanning while staying openable by the recipient.	No

There is deliberately no general PDF rule here. The seven PDF markers in section 7 are reported separately, so a case says "/OpenAction and /Launch were present" rather than collapsing both into one match that says less.

You can add your own

The console has a detection-configuration panel with endpoints for adding rules, NLP patterns and indicators, testing a rule against a sample before committing it, and reloading without a restart. Operator-written rules sit in the CUSTOM family at cap 0.30 - deliberately comparable to a built-in family, because a rule you wrote describes a threat you are actually facing.

9. Optional open-source tools, and the cost of not having them

Four external open-source tools are used when present. All four are free. None is required, and the system never pretends a missing tool did its job - the console reports which are available, and for each absent one, what is specifically not being checked.

Tool	What it adds	If it is missing
TShark (Wireshark)	Reports whether this machine went on to connect to a host that a suspicious email pointed at - turning a suspicion about a message into an observation about what happened next.	No connection evidence. Everything else is unaffected. Needs Npcap and administrator rights on Windows.
oletools (olevba)	Extracts the actual macro source from an Office attachment and reports what would run it automatically.	The built-in inspector still detects that macros are present and whether they auto-run; what is lost is the macro source text in the report.
YARA	Runs the twelve rule files with the reference engine.	Nothing is lost. The built-in engine runs the same rule files. The case records which engine produced the match.
ClamAV	Scans attachment bytes against a signature database held on this machine.	No signature verdict on attachments. Structural inspection and the rules are unaffected.

Why detection had to be written rather than wrapped

On Windows the Wireshark installer leaves PATH untouched, so a tool that is plainly installed is invisible to a naive "is it on PATH" check. The detector therefore looks in the real installation locations, probes Python distributions by importing the module and reading its package metadata, and - critically - reports *why* a tool is unavailable rather than only that it is. "Not installed" and "installed but not on PATH" need completely different things done about them.

What is deliberately not used

- **No paid threat intelligence.** Commercial feeds would be the easy way to raise detection rates and would break the free commitment permanently.
- **No cloud sandbox.** Uploading your attachments to a third party to be detonated sends your documents off your machine. Structural inspection happens locally instead.
- **No model API.** Nothing is sent to a language model for judgement. The language analysis is local and explainable, which is also why its findings can be quoted in the case file.
- **No telemetry.** Your installation reports nothing to anyone, including us.

10. After the verdict: policy, remediation, audit

A verdict that produces no action is just an opinion. Once the confidence engine has decided, the policy engine converts that into an intended action, and the remediation layer carries it out against the real mailbox where a connection exists.

Policy decisions

The policy engine resolves to quarantine the message, alert the recipient, or do nothing - taking into account the verdict, the confidence, whether a protected person is involved, and the posture configured for the sending domain.

Guardrails

- **A never-contain list.** Addresses and domains that must never be quarantined regardless of score, so an automated action cannot take out something critical.
- **Approval gates.** Actions can require explicit human approval before executing; pending actions queue in the console.
- **Rollback.** Executed remediation can be reversed - a released message returns to the mailbox.
- **A containment guard and a correlation guard** that bound what automated response is permitted to do on the strength of correlated rather than direct evidence.
- **Honest failure.** A message with no mailbox provenance is reported as "not actionable" rather than "action failed" - the distinction between "we could not reach it" and "we tried and it broke".

The audit log

Every ingestion, every verdict, every action and every approval is logged with its case id and a SHA-256 of the raw message. The console has a timeline view over it. This is what makes an automated action defensible after the fact.

Reports

A per-case forensic PDF can be generated from the console (GET /api/reports/pdf/:id), containing the verdict, the full evidence breakdown, the relay reconstruction and the indicators.

11. The console

Twenty-three panels. The ones worth knowing about:

Panel	What it is for
Operations overview	Volume, verdict distribution, what is being watched and what is not.
Threat feed	Cases as they arrive, filterable by verdict.
Case investigation	One message in full - every piece of evidence, the confidence breakdown, the relay timeline, the indicators, the attachments.
Evidence graph	The campaign graph rendered: how this case connects to others.
Global threat map	Described below.

Panel	What it is for
Detection depth	Which checks ran, which did not, and why - the honesty surface.
Mail sources	The seven channels, their state, and how to enable the ones that are off.
Response posture	Remediation settings, the never-contain list, per-domain posture.
Admin quarantine	The queue, with release and confirm-threat actions.
Audit timeline	The full log, chronologically.
Check an email	Paste or upload a message for immediate analysis.
Executive view	Protected people, organisation domains, and targeting of them.

The global threat map

Satellite imagery of where message origins resolve, over the whole world - Leaflet with Esri World Imagery, a roads overlay, and an OpenStreetMap layer underneath.

Two things about it are worth stating plainly, because both were learned by measuring rather than assuming:

- **It tells you how old the imagery is.** The caption reads the capture date and resolution for the tile you are looking at, from the imagery provider's own metadata service, and states the age. This matters because the imagery is not uniform: cities are recent and resolve to street level, deserts are coarser, ocean and ice are about 15 metres per pixel, and parts of Siberia have nothing newer than 2012. The map says so rather than letting you assume everything is current.
- **Nowhere is blank.** Where the provider has no imagery it returns a placeholder tile; those are detected by size and made transparent so the OpenStreetMap layer beneath shows through. Verified over the Pacific, Greenland and Antarctica.

Loading was measured and reduced: country view about 1290 ms and street view about 1685 ms, against roughly 2350 ms before. Most of that came from not drawing street geometry over a whole country, where it is a grey haze that costs 900 ms and shows nothing - the roads overlay now starts at zoom 9.

An honest note about coverage

Imagery resolution is the provider's, and it cannot be made uniform by us. It is not the case that any one region is better covered than another by our choice - New York resolves deeper than anywhere in India, and both are limited by what was flown, not by configuration. Serving three imagery hosts in parallel was tried and measured as not faster, so it was removed rather than kept as a plausible-sounding optimisation.

12. The browser extension

The extension exists because the earliest point at which most people can intercept phishing is the inbox in their browser, and because it needs no credential of any kind.

How it reads mail without a password

It runs inside the Gmail or Outlook Web page you are already signed in to. It watches the message list, and for each message it fetches the original using the session key already present in the page. There is no OAuth flow, no app password, no token stored anywhere. If you sign out, it stops working - which is the correct behaviour.

It also refuses a sign-in page handed back in place of a message. When a session lapses the endpoint answers with HTML, and parsed as mail that HTML has no headers at all - so it would be analysed as a message that failed every authentication check, producing a false alarm manufactured entirely by us. The reader checks that what came back is actually a message before submitting it.

What the popup tells you

The popup opens short - the message list and the buttons sit behind a drawer you open if you want them. It shows counts for high-risk, suspicious and legitimate, and each count is a button that filters the list.

The top line is the important one. When nothing is being watched it names **which** of eight cases applies: the permission is missing, no mail tab is open, a mail tab is open but unwatched, it is watching, it is examining, the row selector no longer matches, or the backend rejected the submission - with the reason. "Not reporting yet" is true and useless; these need different things done about them, so they are distinguished.

There is a refresh control that forces a sweep immediately. It does three things rather than one: it clears any backoff from an earlier failure, it asks any running watcher to sweep now, and it starts a watcher in mail tabs that have none - which is the case it most often exists for, since Chrome does not inject content scripts into tabs that were already open when an extension reloaded.

13. The technology stack, in full

Exact versions, read from the lockfiles on 22 September 2026.

Backend runtime

Package	Version	Used for
express	5.2.1	The HTTP API and the static console host.
mailparser	3.9.28	MIME parsing - headers, parts, attachments.
sqlite3	6.0.1	The persistent campaign graph across cases.
smtp-server	3.19.7	The inline SMTP gateway listener.
imap	0.8.19	The IMAP mailbox poller.
nodemailer	10.0.10	Mailbox actions during remediation.
axios	1.20.0	Outbound lookups to free public services.
jsqr	1.4.0	Local QR decoding from attached images.
pdfkit	0.20.2	Per-case forensic report generation.
cors	2.8.6	Origin control for the local API.

Console

Package	Version	Used for
react / react-dom	19.2.8	The interface.
vite	8.2.2	Build and development server.
leaflet	1.9.4	The map engine.
react-leaflet	5.0.0	React bindings for it.
recharts	3.10.1	Charts across the analytics panels.
lucide-react	1.40.0	Icons.
typescript	6.0.2	Type checking in the build.

Desktop and extension

Component	Version	Notes
electron	44.4.3	The desktop shell.
electron-builder	26.15.3	Packages the Windows NSIS installer.
Node.js	24.11.1	The backend runtime.
Extension	Manifest V3	Service worker, content scripts for Gmail and Outlook Web, popup and options pages. Chrome and Edge.

Development and test

`node:test` (the runtime's own test runner, no framework dependency), `jsdom 29.1.1` for running the extension pages and content scripts against a real DOM, `nodemon 3.1.14`, `pdf-parse 2.4.5` and `qrcode 1.5.4` for building test fixtures.

14. Your data: where it lives and what leaves the machine

On disk

`%APPDATA%\phishlens-desktop\data\`

Cases, the deduplication ledger, the thread index, sender baselines, the audit log, the campaign graph database, the channel configuration and the stored secrets all live in that one directory. Nothing is written anywhere else, and nothing is synchronised.

Channels are enabled by editing `channels.json` in that directory, which the desktop supervisor reads at start-up. Note that they are *not* enabled by environment variables - credentials do not go into the Windows registry.

What goes out, and to whom

Leaves the machine	Goes to	Why
Domain and IP indicators	Free public DNS, RDAP and reputation services	Registration age, netblock classification, blocklist presence. Indicators only - never message content.
Map tile requests	The imagery provider	Only the tiles for the area on screen. No case data is in the request.
Message bodies, attachments, headers	Nowhere	Never transmitted. All parsing, inspection and rule evaluation happen locally.
Telemetry, usage data, crash reports	Nowhere	None is collected. We operate no service your copy contacts.

Access control

The API requires a key, generated on first run and held in the data directory. The backend binds to localhost. There is an organisation model with invite-based joining for multi-user installations, and Google sign-in is available for identity only - it is not used to read mail.

15. Running it, testing it, and what is still open

Running it

Install the desktop application and start it. It starts the backend, serves the console and generates the API key. The SMTP gateway listens on port 2525 when enabled.

For the browser channel: load the extension in Chrome or Edge, open its options, paste the API key shown in the console, then open Gmail. If the popup reports nothing being watched, read its top line - it names the specific cause rather than making you guess.

Testing

323 backend checks, 33 desktop, 4 console - all passing. Two of those suites exist because of specific bugs that reached a user, and are worth describing because they say something about how this is built:

- The extension pages are loaded in a headless DOM the way a browser loads them - real HTML, every script, in order. This exists because a popup once called a function that was never defined. A syntax check passes an undefined function happily; only running the line catches it.

- The webmail readers are run against Gmail-shaped markup. This exists because the reader looked for the message id on the row element while Gmail puts it on a span inside the row, so a 2,241-message inbox was read as empty. A reader that quietly stops matching produces no errors at all - it looks exactly like an inbox with no mail in it.

What is finished

Area	State
Detection pipeline	Complete. All 16 stages, 11 evidence families, fusion and scoring.
SMTP gateway channel	Verified end to end - a HIGH_RISK verdict at 0.70 produced through port 2525.
REST, webhook, file upload	Available to any authenticated caller.
Attachment inspection	Complete , including the four decisive findings.
Detection rules	12 rules , running on the built-in engine with or without YARA installed.
Console	23 panels , both themes, error-isolated so one failing panel cannot blank the application.
Global map	Complete - worldwide, gap-free, and it states the imagery age.
Remediation and audit	Complete , with guardrails, approval gates and rollback.

What is open

Item	Status, stated honestly
Browser watcher	The extension is installed and its pages run correctly, but the backend is receiving zero examined and zero failures from it - nothing is arriving at all. The backend answers, the key authenticates, and both the content script and the service worker load cleanly in test. The most likely cause is a mail tab that was open before the extension reloaded; the refresh control in the popup is built precisely for that. The next step is reading the popup's top line, which names the cause - not guessing between the possibilities.
IMAP poller	Needs a Gmail app password, which is yours to generate and enter. It is not something this side should handle on your behalf.
Gmail API channel	Needs a Google Cloud OAuth client created under your own account.
Live packet capture	TShark integration is written but unverified against real capture - it needs Npcap and administrator rights, which is your decision to make.

16. Glossary

Term	Meaning
SPF / DKIM / DMARC	The three email authentication standards. SPF says which servers may send for a domain; DKIM is a cryptographic signature over the message; DMARC says what to do when either fails.
ARC	RFC 8617. A chain of seals recording who handled a message, used here to distinguish forwarding from spoofing - never as evidence of trustworthiness.
ThreatObject	The single object representing one message, which every pipeline stage adds to.
Evidence family	A group of related findings, capped as a group so correlated facts cannot compound into a score they do not deserve.
Decisive finding	A finding with no benign interpretation, which raises a case to 0.70 on its own - visibly, as a named line in the breakdown.
IOC	Indicator of compromise. An IP, domain, URL or file hash extracted from a message.
BEC	Business email compromise. Impersonating a colleague or supplier to redirect a payment or extract information, usually with no malicious link or attachment at all.

Term	Meaning
HTML smuggling	Assembling a file in the browser's memory so no file ever crosses the network to be scanned.
BODY_ONLY	A case built from what was visible in a message list, without full headers. Marked as such so nothing treats it as a complete analysis.
MQL	The rule-expression style the native rule engine uses for detection logic that cites its own source.

Everything in this document was read from the repository at version 1.7.10 on 22 September 2026. Where a thing is not working, it is written here as not working. Counts, versions and constants are quoted from the source rather than remembered.